CSS Flexbox Flex-Basis

Complete Guide: From Basics to Advanced

Contents

Chapter 1

Introduction

1.1 What is flex-basis?

The `flex-basis` property sets the default size of a flex item before any remaining space is distributed. It defines the base size along the main axis, before growing or shrinking is applied.

Key Point: `flex-basis` is the starting point for an item's size. It acts like width (for row) or height (for column) but is overridden by explicit width/height if they're more specific.

1.2 Why Use flex-basis?

- Set default item sizes clearly
- Control minimum sizes for items
- Create responsive grids easily
- Combine with flex-grow for flexibility
- Simplify responsive design
- Avoid min-width/max-width complexity
- Create proportional layouts

1.3 The Flex Properties

`flex-basis` is part of the flex system:

Property	**Purpose**
flex-basis	Default/base size
flex-grow	Growth when space available
flex-shrink	Shrinking when space limited
flex	Shorthand for all three

1.4 flex-basis vs. Width

Understanding the difference is crucial:

flex-basis	**width**
Base size in flex	Fixed size always
Can be overridden	Takes precedence
Responsive by nature	Static value
Works with flex properties	Works independently

Chapter 2

Understanding flex-basis

2.1 How flex-basis Works

```
/* flex-basis sets the base size */
.flex-item {
    flex-basis: 200px;
}

/* Width takes precedence if set */
.flex-item-width {
    width: 200px;
    flex-basis: 300px;  /* Ignored, width wins */
}

/* flex-basis is used when no width */
.flex-item-basis {
    flex-basis: 200px;  /* Used as size */
}

/* With growing/shrinking */
.flex-item-grow {
    flex-basis: 200px;
    flex-grow: 1;  /* Grows from 200px basis */
}
```

2.2 flex-basis Values

2.2.1 Valid Values

- `auto` - Size based on content (default)
- Length units: `200px`, `10em`, etc.
- Percentages: `25%`, `50%`
- `0` - Start from zero for equal distribution
- `content` - Size to content

2.2.2 Examples

```
/* Different flex-basis values */
.fixed { flex-basis: 200px; }      /* Fixed 200px */
.percentage { flex-basis: 50%; }   /* Half container */
.content { flex-basis: auto; }     /* Content size */
.zero { flex-basis: 0; }           /* Grow from zero */
```

2.3 Cascade of Sizing

When sizing flex items, the browser applies:

1. Explicit width/height (highest priority)
2. flex-basis (if no width/height)
3. Content size (lowest priority)

Chapter 3

Core Values

3.1 flex-basis: auto (Default)

Item sizes based on its content or width property.

3.1.1 Syntax

```
.flex-item {
    flex-basis: auto;  /* Default */
}
```

3.1.2 Characteristics

- Uses content size
- Respects width property if set
- Dynamic and responsive
- Default behavior

3.1.3 Examples

Example 1: Content-Sized Items

```
<div class="container">
    <div class="item">Short</div>
    <div class="item">This is longer content</div>
    <div class="item">X</div>
</div>

.container {
    display: flex;
    gap: 10px;
    width: 600px;
}

.item {
    flex-basis: auto;  /* Size to content */
```

```
    background: #3498db;
    padding: 20px;
}

/* Items vary in width based on content */
```

3.2 flex-basis: 0

Item starts from zero and grows based on flex-grow.

3.2.1 Syntax

```
.flex-item {
    flex-basis: 0;
    flex-grow: 1;
}
```

3.2.2 Characteristics

- Perfect for equal distribution
- Ignores content size
- All items grow equally from zero
- Creates equal-width items
- Very useful for grids

3.2.3 Examples

Example 1: Equal Width Columns

```
<div class="columns">
    <div class="column">Column 1 with content</div>
    <div class="column">Column 2</div>
    <div class="column">Column 3</div>
</div>

.columns {
    display: flex;
    gap: 20px;
}

.column {
    flex: 1 1 0;  /* flex-basis: 0 */
    background: white;
    padding: 20px;
}

/* All columns equal width */
```

Example 2: Card Grid

```
<div class="card-grid">
    <div class="card">Card 1</div>
    <div class="card">Card 2</div>
    <div class="card">Card 3</div>
    <div class="card">Card 4</div>
</div>

.card-grid {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}

.card {
    flex: 1 1 0;
    min-width: 200px;  /* Minimum width */
    background: white;
    padding: 20px;
}

/* Cards equal width, wrap when needed */
```

Example 3: Button Group

```
<div class="button-group">
    <button>Save</button>
    <button>Edit</button>
    <button>Delete</button>
</div>

.button-group {
    display: flex;
    gap: 10px;
}

.button-group button {
    flex: 1 1 0;
    padding: 12px;
    background: #3498db;
    color: white;
    border: none;
    cursor: pointer;
}

/* All buttons equal width */
```

3.3 flex-basis: 200px (Fixed Size)

Item has a specific base size.

3.3.1 Syntax

```
.flex-item {
    flex-basis: 200px;
}
```

3.3.2 Characteristics

- Specific starting size
- Can grow/shrink from this base
- Useful for minimum widths
- Clear, predictable sizing

3.3.3 Examples

Example 1: Sidebar Layout

```
<div class="layout">
    <aside class="sidebar">Sidebar</aside>
    <main class="content">Main Content</main>
</div>

.layout {
    display: flex;
    gap: 20px;
}

.sidebar {
    flex-basis: 250px;
    flex-grow: 0;
    background: #ecf0f1;
}

.content {
    flex-basis: 0;
    flex-grow: 1;
    background: white;
}

/* Sidebar 250px, content flexible */
```

Example 2: Responsive Grid

```
<div class="product-grid">
    <div class="product">Product 1</div>
    <div class="product">Product 2</div>
    <div class="product">Product 3</div>
</div>

.product-grid {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
```